

Great. Below are the **Top 50 "Why?" interview questions** that senior interviewers ask after you explain your GenAI project. These are common in **3-8 years GenAI Engineer** interviews.

1. Why did you choose LangChain?

Answer:

"LangChain provides ready-made components for document loading, chunking, embeddings, vector stores, retrievers, prompt templates, and LLM integration. It reduces development time and makes building RAG applications easier."

2. Why not use LlamaIndex?

Answer:

"LlamaIndex is excellent for data indexing and retrieval, but LangChain provides broader support for workflows, chains, agents, and integrations. Since my project required an end-to-end RAG pipeline, LangChain was a better fit."

3. Why did you choose FAISS?

Answer:

"FAISS is open source, fast, lightweight, and runs locally. It is ideal for development and small to medium-sized RAG applications."

4. Why not Pinecone?

Answer:

"Pinecone is a managed cloud vector database designed for production-scale systems. My project did not require distributed storage or managed infrastructure, so FAISS was sufficient."

5. Why did you use Hugging Face embeddings?

Answer:

"They provide high-quality semantic embeddings, support many languages, run locally, and avoid external API costs."

6. Why not OpenAI embeddings?

Answer:

"OpenAI embeddings are excellent but require API calls and incur usage costs. For a local or offline application, Hugging Face or Ollama embeddings are more suitable."

7. Why use Ollama?

Answer:

"Ollama allows local execution of open-source LLMs, eliminating API costs while improving privacy and reducing internet dependency."

8. Why choose Llama 3?

Answer:

"Llama 3 provides strong reasoning, good instruction following, and high-quality responses while remaining efficient enough for many local deployments."

9. Why not GPT?

Answer:

"GPT offers excellent performance but depends on cloud APIs and associated costs. Llama 3 can be deployed locally, which was more appropriate for my project."

10. Why use RAG?

Answer:

"RAG enables the model to answer questions using external documents without retraining the LLM, reducing hallucinations and keeping information up to date."

11. Why not Fine-tuning?

Answer:

"Fine-tuning changes model behaviour but doesn't continuously incorporate new documents. RAG is more practical for frequently updated knowledge bases because new documents can be indexed without retraining."

12. Why split documents?

Answer:

"Large documents exceed the model's context window. Chunking makes embedding generation, retrieval, and prompt construction more efficient."

13. Why use chunk overlap?

Answer:

"Overlap preserves context between adjacent chunks, preventing important information from being lost when text is split."

14. Why choose 500 chunk size?

Answer:

"It provides a balance between maintaining context and retrieving focused, relevant information."

15. Why not 1000?

Answer:

"Larger chunks often include unrelated content, reducing retrieval precision."

16. Why not 200?

Answer:

"Very small chunks may lose important context, resulting in incomplete answers."

17. Why convert text into embeddings?

Answer:

"Embeddings transform text into numerical vectors that allow semantic similarity search."

18. Why can't SQL search do this?

Answer:

"Traditional SQL keyword searches match exact words, whereas embeddings enable semantic search based on meaning."

19. Why use cosine similarity?

Answer:

"Cosine similarity compares the direction of vectors rather than their magnitude, making it well suited for semantic embeddings."

20. Why Top-K = 5?

Answer:

"Five retrieved chunks generally provide enough context without introducing excessive irrelevant information."

21. Why not Top-K = 20?

Answer:

"Retrieving too many chunks increases prompt size, cost, and the likelihood of irrelevant context."

22. Why use Prompt Templates?

Answer:

"Prompt templates ensure a consistent prompt structure and make prompts easier to maintain."

23. Why use System Prompts?

Answer:

"System prompts define the assistant's behaviour and help reduce hallucinations."

24. Why use Metadata?

Answer:

"Metadata enables filtering by document, page, author, department, date, or user, improving retrieval relevance."

25. Why use a Retriever?

Answer:

"A retriever efficiently fetches only the most relevant document chunks rather than searching every document manually."

26. Why use FastAPI?

Answer:

"FastAPI provides high performance, asynchronous support, automatic API documentation, and easy integration with AI services."

27. Why not Flask?

Answer:

"Flask is lightweight but requires more manual setup. FastAPI offers better performance and built-in features for APIs."

28. Why use Streamlit?

Answer:

"Streamlit allows rapid development of AI demos and internal tools with minimal frontend code."

29. Why use Docker?

Answer:

"Docker packages the application and its dependencies into a portable, reproducible environment."

30. Why Kubernetes?

Answer:

"Kubernetes provides orchestration, auto-scaling, load balancing, rolling updates, and self-healing."

31. Why cache embeddings?

Answer:

"Embeddings are deterministic for the same input, so caching avoids unnecessary recomputation."

32. Why cache prompts?

Answer:

"Caching repeated prompts reduces latency and lowers LLM usage costs."

33. Why re-ranking?

Answer:

"Re-ranking improves retrieval quality by reordering candidate chunks based on deeper relevance."

34. Why Hybrid Search?

Answer:

"Hybrid search combines keyword matching and semantic search to improve retrieval performance."

35. Why OCR?

Answer:

"Scanned PDFs contain images instead of selectable text, so OCR extracts text before processing."

36. Why ANN?

Answer:

"Approximate Nearest Neighbour algorithms provide fast similarity search over very large vector collections."

37. Why HNSW?

Answer:

"HNSW offers high-speed vector search with excellent recall, making it popular for production systems."

38. Why monitor token usage?

Answer:

"Token usage directly affects cost, latency, and context window limits."

39. Why monitor latency?

Answer:

"High latency impacts user experience and may indicate retrieval or inference bottlenecks."

40. Why monitor hallucinations?

Answer:

"Monitoring helps identify when the model generates unsupported or inaccurate information."

41. Why evaluate retrieval?

Answer:

"Poor retrieval leads to poor answers, even if the LLM itself is very capable."

42. Why RAGAS?

Answer:

"RAGAS provides automated metrics such as faithfulness, answer relevance, and context precision for evaluating RAG systems."

43. Why LangSmith?

Answer:

"LangSmith helps trace requests, inspect prompts, debug chains, and evaluate application performance."

44. Why use Agents?

Answer:

"Agents enable the LLM to choose tools dynamically and perform multi-step reasoning."

45. Why use MCP?

Answer:

"MCP standardizes how LLMs interact with external tools and data sources, improving interoperability."

46. Why not store PDFs directly in the LLM?

Answer:

"LLMs don't permanently store uploaded documents. RAG retrieves relevant content from an external knowledge base whenever a user asks a question."

47. Why regenerate embeddings when documents change?

Answer:

"Embeddings represent the document content. When the content changes, the embeddings must be updated to reflect the new information."

48. Why not rebuild the entire vector database?

Answer:

"Updating only the modified documents is more efficient and avoids unnecessary computation."

49. Why use asynchronous processing?

Answer:

"Asynchronous processing allows long-running tasks like document ingestion and embedding generation to run without blocking user requests."

50. Why should we hire you for a GenAI Engineer role?

Best Interview Answer

"I understand both the theory and the implementation of GenAI systems. I've built end-to-end RAG applications involving document loading, chunking, embeddings, vector databases, similarity search, prompt engineering, and LLM integration. I can debug retrieval issues, optimize performance, reduce hallucinations, and deploy applications using tools like FastAPI, Docker, and Kubernetes. Beyond using frameworks, I understand the design decisions behind them and can explain the trade-offs."

★ Most Important Questions (Asked in Almost Every Interview)

If you can confidently answer these **10 questions**, you'll be well prepared for many GenAI technical interviews:

1. Explain your complete RAG pipeline.
2. Why did you choose that chunk size?
3. Why FAISS instead of Pinecone?
4. Why RAG instead of Fine-tuning?
5. How do you reduce hallucinations?
6. How do you improve retrieval accuracy?
7. What happens if you change the embedding model?
8. How do you debug incorrect answers?
9. How do you deploy a GenAI application?
10. What was the biggest challenge in your project, and how did you solve it?

Mastering these answers, along with the earlier concepts and scenario questions, provides a strong foundation for most GenAI Engineer interviews.

yes

Excellent. Below are **20 advanced scenario-based GenAI interview questions** that are commonly asked by companies such as TCS, Cognizant, Accenture, Capgemini, IBM, Deloitte, Infosys, Wipro, HCL, Amazon, Microsoft, and startups for **3-8 years of experience**.

Scenario 1: Wrong Answer

Interviewer

Your RAG chatbot gives the wrong answer. How will you debug it?

Answer

"I debug the pipeline step by step instead of assuming the LLM is the problem.

1. Check the user query.
2. Verify document loading.
3. Inspect generated chunks.
4. Verify embeddings.
5. Check retrieved Top-K chunks.
6. Verify similarity scores.
7. Inspect the final prompt.
8. Check the LLM response.

In most cases, incorrect answers are caused by poor retrieval rather than the LLM."

Scenario 2: User uploads 5000 PDFs

Interviewer

How would you process them?

Answer

"I would not process them synchronously. I would use a background queue, batch embedding generation, parallel workers, and incremental indexing into the vector database."

Scenario 3: Vector DB contains 50 million vectors

Interviewer

How will you optimize search?

Answer

"I would use ANN indexing such as HNSW or IVF, metadata filtering, vector compression, caching, and distributed vector databases."

Scenario 4: Users ask the same question repeatedly

Answer

"I would cache the retrieved context or final response so repeated requests avoid unnecessary embedding generation, vector search, and LLM inference."

Scenario 5: Documents change every day

Interviewer

Will you rebuild everything?

Answer

"No. I would identify changed documents, regenerate embeddings only for those documents, update the vector database, and remove outdated vectors."

Scenario 6: PDF contains only images

Answer

"I would perform OCR using Tesseract or a cloud OCR service, then continue with chunking, embedding generation, and indexing."

Scenario 7: LLM ignores retrieved context

Answer

"I would strengthen the system prompt, ensure retrieved chunks are included in the prompt, reduce irrelevant retrieved context, and instruct the model to answer only from the provided documents."

Scenario 8: Retrieval quality is poor

Answer

"I would evaluate chunk size, overlap, embedding quality, metadata filters, Top-K value, and whether re-ranking is needed."

Scenario 9: Customer uploads a 2 GB PDF

Answer

"I would stream the file, process it page by page, generate embeddings in batches, and avoid loading the entire file into memory."

Scenario 10: API cost is very high

Answer

"I would reduce prompt size, retrieve fewer chunks, cache responses, summarize long documents, and use smaller models when suitable."

Scenario 11: Multiple users upload documents

Interviewer

How do you isolate them?

Answer

"Each user gets a separate namespace or collection. During retrieval, I filter by user ID so users can access only their own documents."

Scenario 12: A customer asks questions in Tamil while documents are in English

Answer

"I would use multilingual embedding models that map different languages into a shared semantic space or translate the query before retrieval."

Scenario 13: Hallucination

Interviewer

How will you reduce hallucinations?

Answer

"I would improve retrieval quality, use trusted documents, add metadata filtering, strengthen the system prompt, use re-ranking, and instruct the model to respond with 'I don't know' if the answer is not in the context."

Scenario 14: FastAPI vs Flask

Interviewer

Why FastAPI?

Answer

"FastAPI provides asynchronous processing, better performance, automatic OpenAPI documentation, and data validation using Pydantic."

Scenario 15: Why not Fine-tune?

Answer

"Our documents change frequently. RAG allows new knowledge to be indexed without retraining the model, making updates faster and more cost-effective."

Scenario 16: User uploads duplicate PDFs

Answer

"I would detect duplicates using file hashes or content hashes and avoid regenerating embeddings for identical documents."

Scenario 17: Retrieval takes 5 seconds

Answer

"I would profile the system, optimize ANN indexes, reduce Top-K, enable caching, improve hardware, and parallelize expensive operations."

Scenario 18: Production monitoring

Interviewer

What metrics will you monitor?

Answer

"Latency, token usage, retrieval accuracy, hallucination rate, cache hit rate, error rate, CPU/GPU utilization, and user feedback."

Scenario 19: Prompt Injection

Interviewer

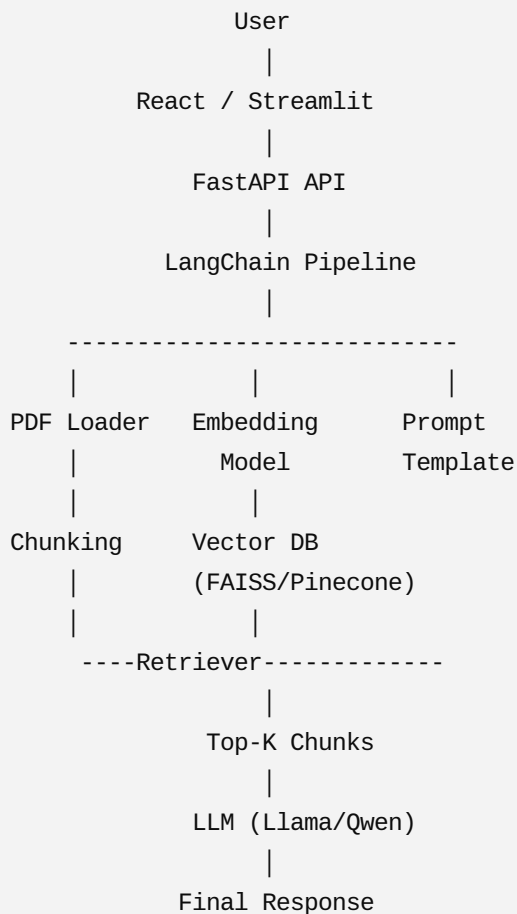
How will you stop it?

Answer

"I would validate user input, isolate system prompts, restrict tool permissions, sanitize retrieved content, and implement guardrails."

Scenario 20: Explain your project architecture

Answer



★ Ultimate Interview Question (Most Difficult)

Interviewer

Suppose I remove FAISS from your project. Can your chatbot still work?

Best Answer

"Yes, but it would not scale efficiently. Without FAISS or another vector database, I would have to compare the query embedding with every document embedding sequentially (a brute-force search). That approach works for very small datasets but becomes slow and resource-intensive as the number of documents grows. A vector database provides efficient indexing and fast similarity search, which is essential for production-scale RAG systems."

★ The Hardest Question (Asked by Senior Architects)

Interviewer

What happens internally when a user asks a question? Explain every step.

Best Answer

"When the user submits a question, the application first validates and preprocesses the input. The same embedding model used during document indexing converts the query into a vector. The retriever performs a similarity search against the vector database and returns the Top-K most relevant chunks. Optional metadata filtering or re-ranking improves relevance. These chunks, along with the user's question and system instructions, are assembled into a prompt. The LLM processes the prompt, generates a grounded response, and returns it to the user. Finally, the application logs metrics such as latency, token usage, and retrieval quality for monitoring and future optimization."

🎯 If you're targeting GenAI Engineer interviews, the complete preparation roadmap is:

- **✅ Part 1:** 94 core interview questions (completed)

-  **Part 2:** 30 scenario-based interview questions (completed)
-  **Part 3:** Top 50 "Why?" interview questions (completed)
-  **Part 4:** 20 advanced production scenarios (completed)

A valuable next step would be a **mock interview with 100 questions**, where I play the interviewer and ask questions in the order they are typically asked in real technical rounds—from basic concepts to advanced architecture, debugging, coding, and production scenarios. This closely simulates an actual GenAI Engineer interview.

yes

Excellent. Let's do a **real GenAI Engineer Mock Interview (100 Questions)**.

I'll ask questions exactly like an interviewer. After each question, I'll provide the **expected answer** and **follow-up questions** that interviewers commonly ask.

Round 1 - Introduction (Questions 1-10)

Question 1

Tell me about yourself.

Expected Answer

"I am a Generative AI Engineer with experience building RAG-based applications using Python, LangChain, Hugging Face/Ollama embeddings, FAISS, and LLMs like Llama and Qwen. I have developed applications that load documents, split them into chunks, generate embeddings, store them in vector databases, retrieve relevant context using similarity search, and generate accurate responses using LLMs. I also have experience with FastAPI, Streamlit, Docker, and cloud deployment."

Follow-up

- Tell me about your latest GenAI project.
- What was your role?

- How many people were on your team?
-

Question 2

Explain your GenAI project from start to finish.

Expected Answer

1. User uploads PDF.
 2. PDF Loader reads document.
 3. Split into chunks.
 4. Generate embeddings.
 5. Store vectors in FAISS.
 6. User asks a question.
 7. Convert question into embedding.
 8. Similarity search.
 9. Retrieve Top-K chunks.
 10. Prompt Template.
 11. LLM.
 12. Final answer.
-

Question 3

Why did you choose RAG?

Expected Answer

"The documents change frequently. Fine-tuning for every update is expensive and slow. RAG allows me to retrieve the latest information without retraining the model."

Question 4

What problems does RAG solve?

Expected Answer

- Hallucination reduction
- Latest knowledge
- Private documents
- Lower cost than fine-tuning

- Faster updates

Question 5

What are the components of RAG?

Expected Answer

Loader

↓

Chunking

↓

Embedding

↓

Vector Database

↓

Retriever

↓

Prompt

↓

LLM

↓

Response

Question 6

Explain document loading.

Expected Answer

"Document loading is the process of reading documents such as PDF, DOCX, TXT, HTML, or CSV into memory using document loaders."

Question 7

What loaders have you used?

Expected Answer

- PyPDFLoader
 - TextLoader
 - CSVLoader
 - DirectoryLoader
 - WebBaseLoader
-

Question 8

Why can't LLMs read PDFs directly?

Expected Answer

"LLMs process text tokens, not document file formats. The content must first be extracted from the file."

Question 9

What happens after loading?

Expected Answer

"The document is split into chunks so that it can be embedded efficiently and fit within the model's context window."

Question 10

What is chunking?

Expected Answer

"Chunking is the process of dividing a document into smaller pieces while preserving enough context for effective retrieval."

Round 2 - Chunking (Questions 11-20)

Question 11

Why is chunking necessary?

Question 12

What happens if we don't chunk?

Question 13

Explain `RecursiveCharacterTextSplitter`.

Question 14

Why chunk overlap?

Question 15

Why 500 chunk size?

Question 16

Why not 1000?

Question 17

Why not 200?

Question 18

What is chunk overlap?

Question 19

How do you decide chunk size?

Question 20

Can chunk size affect accuracy?

Round 3 - Embeddings (Questions 21-35)

21. What are embeddings?
 22. Why embeddings?
 23. How are embeddings generated?
 24. Which embedding model did you use?
 25. Why that model?
 26. What is embedding dimension?
 27. Why same embedding model?
 28. Dense vs Sparse embeddings?
 29. Cosine similarity?
 30. Euclidean distance?
 31. Dot product?
 32. ANN Search?
 33. HNSW?
 34. IVF?
 35. Product Quantization?
-

Round 4 - Vector Database (Questions 36-45)

36. What is FAISS?
37. Why FAISS?
38. Why not Pinecone?
39. Why not Chroma?
40. Why vector DB?
41. How is indexing done?
42. Flat Index?
43. IVF Index?
44. HNSW?

45. Metadata filtering?

Round 5 - Retrieval (Questions 46-55)

- 46.** Similarity Search?
 - 47.** Retriever?
 - 48.** Top-K?
 - 49.** Re-ranking?
 - 50.** Hybrid Search?
 - 51.** Context Precision?
 - 52.** Context Recall?
 - 53.** Hit Rate?
 - 54.** Retrieval Accuracy?
 - 55.** RAGAS?
-

Round 6 - Prompt Engineering (Questions 56-65)

- 56.** Prompt Engineering?
 - 57.** System Prompt?
 - 58.** User Prompt?
 - 59.** Temperature?
 - 60.** Top-p?
 - 61.** Max Tokens?
 - 62.** Hallucination?
 - 63.** Reduce hallucination?
 - 64.** Prompt Injection?
 - 65.** Guardrails?
-

Round 7 - LLM (Questions 66-75)

- 66.** Explain Transformers.
- 67.** Attention?
- 68.** Self Attention?
- 69.** Tokenization?
- 70.** Context Window?
- 71.** Fine-tuning?

- 72. LoRA?
 - 73. Quantization?
 - 74. Inference?
 - 75. Streaming Response?
-

Round 8 - LangChain (Questions 76-85)

- 76. LangChain?
 - 77. Chain?
 - 78. Runnable?
 - 79. PromptTemplate?
 - 80. OutputParser?
 - 81. Memory?
 - 82. Agent?
 - 83. Tool?
 - 84. LangGraph?
 - 85. MCP?
-

Round 9 - Production (Questions 86-95)

- 86. Docker?
 - 87. Kubernetes?
 - 88. FastAPI?
 - 89. Monitoring?
 - 90. Logging?
 - 91. Token Cost?
 - 92. Latency?
 - 93. Caching?
 - 94. Multi-user support?
 - 95. Scaling?
-

Round 10 - Architecture & Debugging (Questions 96-100)

Question 96

Draw your complete architecture.

```
User
|
React / Streamlit
|
FastAPI
|
LangChain
|
Loader
|
Chunking
|
Embeddings
|
FAISS
|
Retriever
|
Prompt
|
LLM
|
Response
```

Question 97

Your chatbot gives wrong answers. How do you debug it?

Expected approach:

- Check document loading.
- Verify chunking.
- Validate embeddings.
- Inspect retrieved chunks.
- Review prompt.
- Evaluate LLM output.

Question 98

What was the biggest challenge in your project?

Example answer:

"Initially, retrieval quality was poor because the chunk size was too large. After tuning chunk size, adding overlap, and introducing metadata filtering, retrieval accuracy improved significantly."

Question 99

Why should we hire you?

Example answer:

"I understand the complete lifecycle of GenAI applications—from document ingestion and retrieval to prompt engineering, deployment, monitoring, and optimization. I can explain design decisions, debug production issues, and build scalable RAG solutions."

Question 100 (Most Difficult)

Explain everything that happens after the user uploads a PDF until the answer is displayed.

Expected Answer

"The application loads the PDF, extracts text, splits it into chunks, generates embeddings, and stores them in a vector database. When the user asks a question, the query is embedded using the same embedding model. The retriever performs similarity search to find the most relevant chunks. Those chunks are combined with the user's question and system instructions in a prompt. The LLM generates an answer grounded in the retrieved context, and the application returns the response while logging metrics such as latency, token usage, and retrieval performance."

Interview Success Tip

If you can confidently answer these 100 questions **and explain the reasoning behind your design choices**, you'll be well prepared for many GenAI Engineer interviews, from fundamentals through production-level discussions. Practice answering aloud in 2–3 minutes per question, and be ready for follow-up questions that ask you to justify your choices with trade-offs rather than just definitions.

ChatGPT can make mistakes. Check important info.